

AIがコードを書く時代の仕様駆動開発

ずんだもんボイスボット での実践

自己紹介・今日話すこと

- ずんだもんボイスボットの開発で試みた「仕様駆動開発」
- AIに実装を任せるために、人間は何を書くべきか

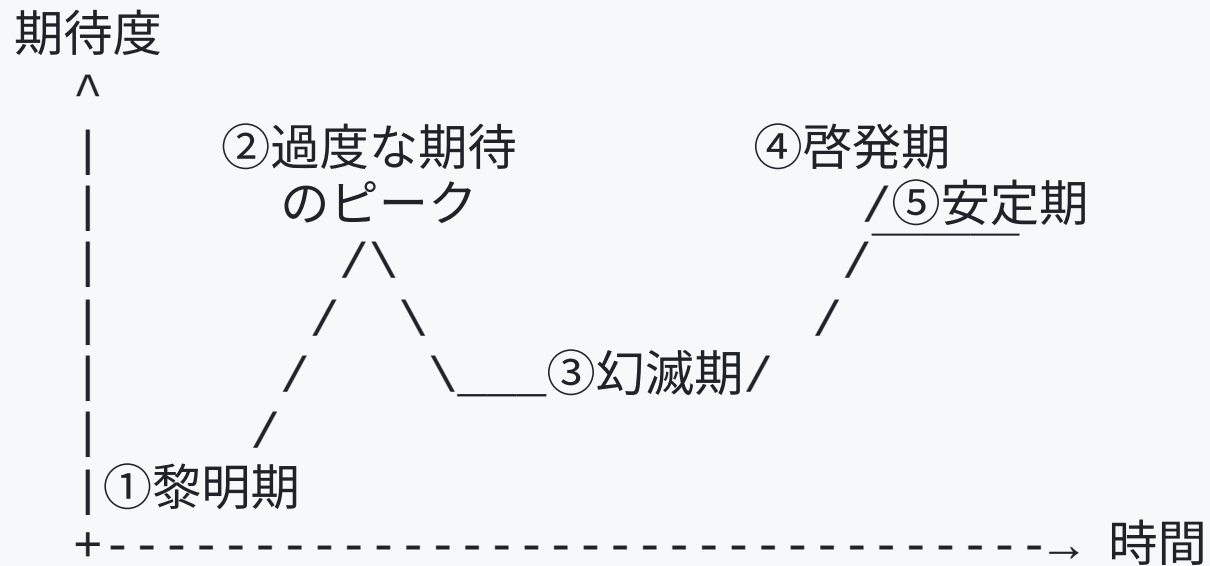
バイブコーディングとは

"I just see things, say things, run things, and copy paste things, and it mostly works."

— Andrej Karpathy (2025)

- AIにコードを書かせ、人間はほとんどコードを読まない開発スタイル
- 「動けばOK」の精神で、雰囲気（vibes）に任せて進める
- プロトタイピングや個人開発では非常に高速

ガートナーのハイプ・サイクルで見ると...



- AI驚き屋（情報商材屋、コンサル）が「工数大幅削減！」と喧伝
- 今は ②過度な期待のピーク期 → これから ③幻滅期 へ
- 「AIに任せたら動かない」「保守できない」という現実直面する時期

バイブコーディングの課題

バイブス（雰囲気）に任せてコードを書き、AIの出力が意図通りに動作すると盲信すること

- ナレッジ管理の欠如
- 出力のズレによる手戻り発生

大きな技術的負債となりうる（下手に動くのでタチがさらに悪い）

AIがコードを書く時代に起きること

- 生成速度が上がる → 意思決定の記録が追いつかない
- 「なぜそう実装したか」がブラックボックス化
- 技術的負債に加えて理解負債という言葉が誕生

Spec-Driven Development (SDD) とは？

AIが実行可能な明確なSpecを書くことが主要な成果物になるアプローチ

→ ハイプ・サイクルの **啓発期** に向かうための実践

最近だとKiroとかが流行っている。

2つの考え方の転換

- 開発の共通言語が「コード」から「自然言語」へ
- 人間は承認役、AIが開発の主体

原則

ドキュメントが正本。コードが矛盾したらドキュメントが勝つ。

実践

- ドキュメント変更がアーキテクチャの Single Source of Truth
- コード変更は、まず仕様変更から始める
- AIもドキュメントに従う

責務境界を「禁止事項」で定義する

許可	禁止
app → session	session → ai（直接呼び出し禁止）
app → ai	http → sip/rtp（触れてはいけない）
session → sip/rtp	media → 配信ロジック

なぜ禁止を明文化するか

- 「やっていいこと」より「やってはいけないこと」の方が明確
- レビューで「これはNG」と即判断できる
- AIにも境界を守らせられる

AI分業：Claude Code vs Codex

観点	Claude Code	Codex
担当	仕様/ドキュメント	実装/テスト
出力	仕様修正案、質問リスト	コード差分
コード変更	しない	する
縛り	CLAUDE.md	AGENTS.md

ポイント

- 実装AIは仕様に従うだけ
- 仕様を握る者がプロダクトを握る

AGENTS.md：実装AIへの指示書

依存方向（必須）

- app → session → (sip, rtp) → transport
- app → ai
- 禁止: session から ai 直呼び

timeout/retry

- 外部 I/O は timeout 必須
- retry は ai モジュールに閉じ込める

Backpressure (MVP)

- RTP → ASR: 最新優先（古い PCM 破棄）
- TTS → RTP: 割り込み優先

チケットゲート：勝手に作らせない

CLAUDE.md より

合意（チケット参照）が確認できない限り、作業を開始しないでください。

効果

- 「とりあえず作っちゃった」が防げる
- 意思決定の記録が必ず残る
- 後から「なぜこれがあるか」を追跡可能

Before / After

Before	After
「このコード誰が決めた？」	design.md のこの章
「なんでこの実装？」	ADR に判断理由
「勝手に機能増えてる」	チケットゲートで防止
「禁止事項？口伝です」	AGENTS.md に明文化
「AIが変なコード書いた」	仕様が曖昧だった証拠

やってみてわかったこと

良かった点

- レビューが「仕様通りか」の確認に集中できる
- 新規参入者が docs からオンボーディングできる
- AIの出力品質が仕様の精度に比例するということがわかった

大変だった点

- 最初に docs を書く労力
- 「とりあえず動かしたい」誘惑との戦い(これは個人開発なのでよく負けてますが。。。)
- docs とコードの同期を保つ運用
- 自分で書いたわけではないので、承認作業が非常に大変

これだったら書いたほうが早いんじゃない？？？？

- 本来のソフトウェア工学に基づいたアジャイル開発に進化しただけ
- 環境を整備すると多分楽になる（その分学習コストが高いが）

AI駆動開発(SDDになるのかな?)の前にやっておきたいこと

- AIがやれること/やれないことを事前に認識しておく。
- AIの構造を理解しておき、効率的に動かす仕組みを理解しておく。
- AIにやりたい放題やられないように、常に監視する仕組みを作る(承認フローなど)。
- 信頼できる有識者、専門家を見つけ、なぜ成功したのかを考察する。
- AI驚き屋に騙されない
- AI驚き屋に騙されない
- AI驚き屋に騙されない

人間が骨組みを考え、作り、品質に対する挑戦を行うというのは常に変わらない

まとめ

AIがコードを書く時代、人間の価値は「何を作るか」を決めること。

- 仕様 = 意思決定の記録
- 仕様駆動 = 意思決定駆動
- 仕様を握る者がプロダクトを握る

AI驚き屋と有識者・専門家の違い

- 前者：投資しなくても実現可能な時代がやってきた、と不安を煽り、LINE誘導またはBrainに誘導する（で薄っぺらい商材買わされて終わり）
- 後者：投資を怠らず（月額3万は安い方、平均10万くらい？）、アーキテクチャも洗練されている